OPTIMISING THE CODE

When I designed the language I never imagined it would become what it has now. One massive problem with the code each time it is read, is to decide ⓐ is it an operand? ⓑ is it an operator?

For checking Operators, I strcmp from operator-00 all the way to the last one, operator-121. This means that on average, the operator will need about 60 x strcmp's function calls! This makes the program run very slow!

When we have an Operand, it's even worse! Every single Operator from operator-00 to operator-121 must be strcmp'd with the current token. When it's not found (after 121 strcmp function calls), ONLY then is it pushed onto the stack.

To avoid this problem, I created a Hashset instead. This would make it a lot quicker as it directly maps all the tokens to a unique checksum, and massively improves performance. We initialise this Hashset on the heap at the begging of the program, before we start cycling through each token.

I made the size of the Hashset array 3 x "size of all operators", so in this case 3 x 121 = 363 resq's. The Hashset must also be nulled out at the start before we add to it.

I also implemented linked lists for all of the resq's in the array to handle any clashing if a checksum is returned twice or more.

On the next page is a short amount of 'C' code to show the logic of how the Hashset works.

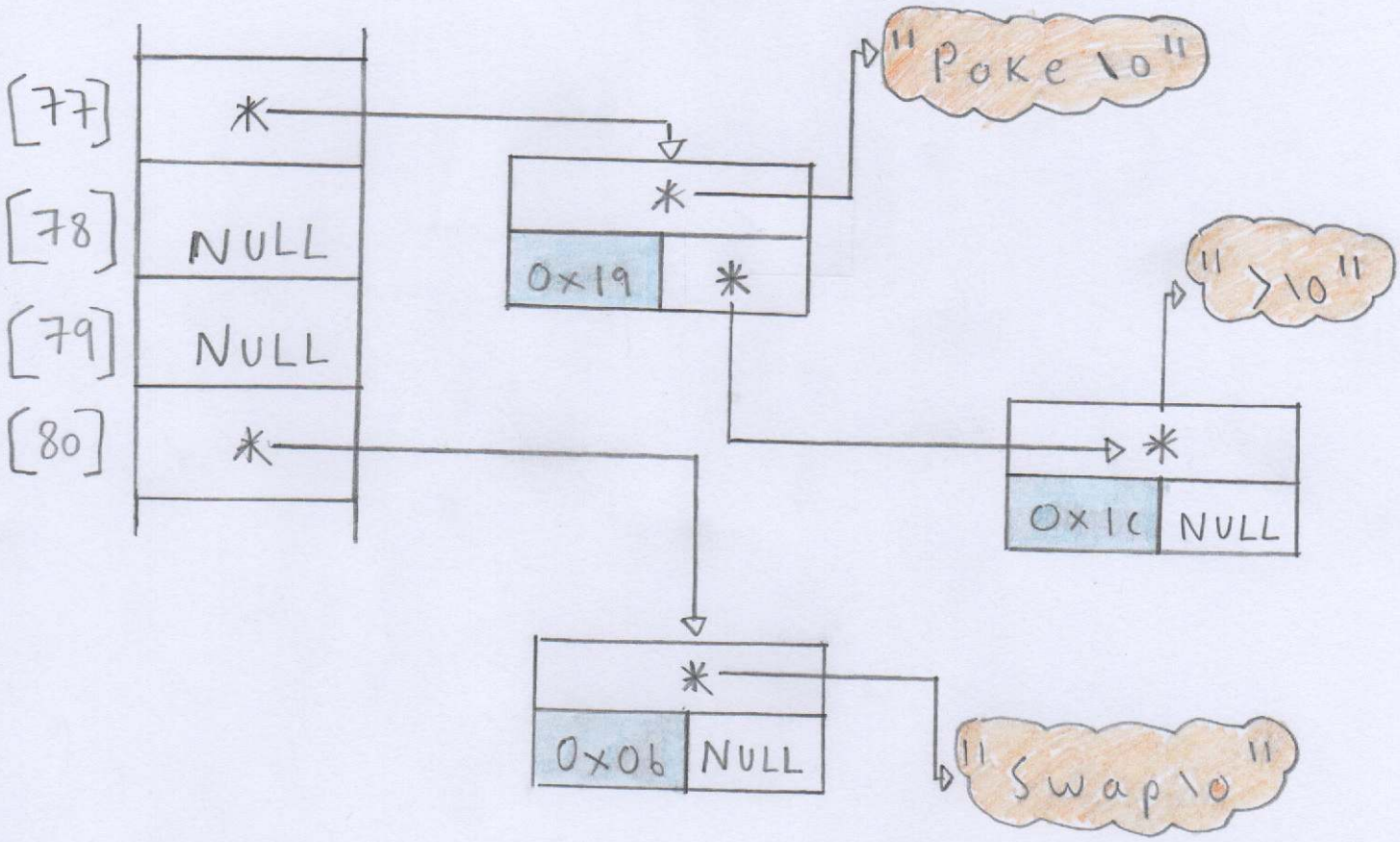
The function below reads a string a char / byte at a time. It then calculates a checksum and % by 363 to find an index. This index is then returned.

```
Const int MAX-HASH-SET = 363;
const int PRIME-NUM = 31;
char *ptr = "abc";

int generateChecksum (char *ptr) {
    int key = 0;
    if (!ptr) {
        return -1;  // error!
    }
    while (*ptr) {
        key += (*ptr++ * PRIME-NUM);
        // now the number is mult by a prime
    }
    return key % MAX-HASH-SET;
    // for x3 list of operators
}
```

In this function, you pass a string and it calculates the [index], in the hashset where to find / add this String. Empty strings returns -1 error!

Below is a diagram of how a section of the Hashset might look and how it handles clashes. Indexes are on the left hand side.



As you can see, each node is made up of 17 bytes.

bytes [0 -> 7] = pointer to name of operator.
bytes [8] = index to use in [map-funcs + (n) x 0x08].
bytes [9 -> 16] = pointer to next node or NULL.

In the above:

"poke" operator will call [map-funcs + 0x19 x 0x08].
">" operator will call [map-funcs + 0x1c x 0x08].
"swap" operator will call [map-funcs + 0x06 x 0x08].

When this program comes across an if / loop statement, it has to find where the start and end braces are. That means that if the body of the if or loop statement has MANY operands and operators inside it, it will do a strcmp on every single one, until it reaches the end brace. Calling an if / loop statement hundreds or even thousands of times would mean thousands and thousands of strcmp's being carried out! This is a huge factor as to why our results were so poor.

So I decided to find all the matching opening and closing braces only once and do this before we start the code. This would be saved in the curly_brace_table. In this table, the 1st resav is the index of the opening '{' brace and the 2nd resav is the index of the closing '}' brace that matches it.

For example:

(?) = operand or operator

```
_start:
(1) { (?) (?) (?)
          (5) { (?) (?) (?) (?) } (10) (?) (?) if
      } (?) (?) (?) loop
     (14)
```

curly_brace_table:

(0)	1	14
(2)	5	10

The matching closing brace index for (1) can be found at (14).
The matching closing brace index for (5) can be found at (10).

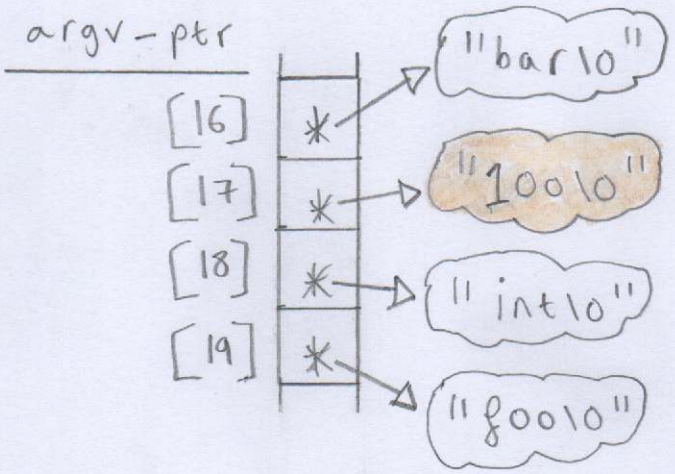
A function to return this value (when given the index of the opening '{' brace) means we can save many unnecessary strcmp's.

When the source code is parsed, all the tokens are saved as strings, even integers. When the "int" operator is seen, it uses the top of the rts and takes this string and converts it to a number, then pops the string off and replaces it with an integer.

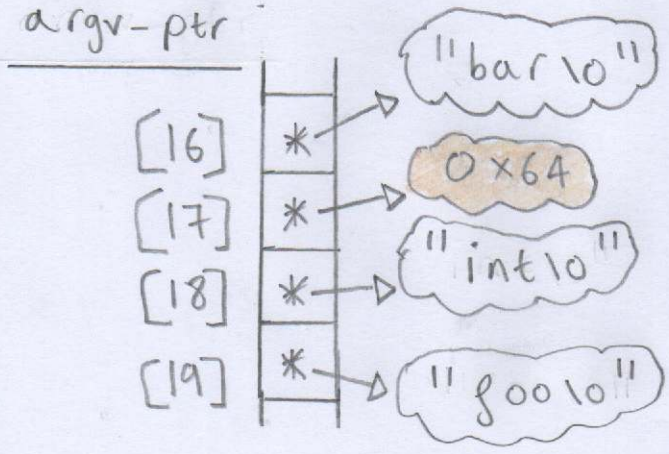
If a function had 10 integers and this function was called thousands of times, this would mean we would have to convert 10 strings into 10 integers thousands of times!

Just like searching for curly braces slowing down the program, this is probably just as bad!

So I decided to convert all integers from string to their number once, before the program starts.



First scan all the tokens from [0] index to the last index in the argv-ptr array. When we come across an "int", we move the pointer -1 index and pass this string "100" to a function that converts it to 0x64. Then free the "100" heap allocation. Allocate a new 8 Bytes on the heap and place 0x64 in this, then insert it to the location -1 index before "int".



Now the argv-ptr array looks like this and we continue to search for more "int" to convert.

The "int" operator now simply dereferences the value at the top of the rts and replaces it with the value it had contained in the heap allocation.

The Hashset made running the code faster by 500-700%, this was good, but I had another idea to improve performance!

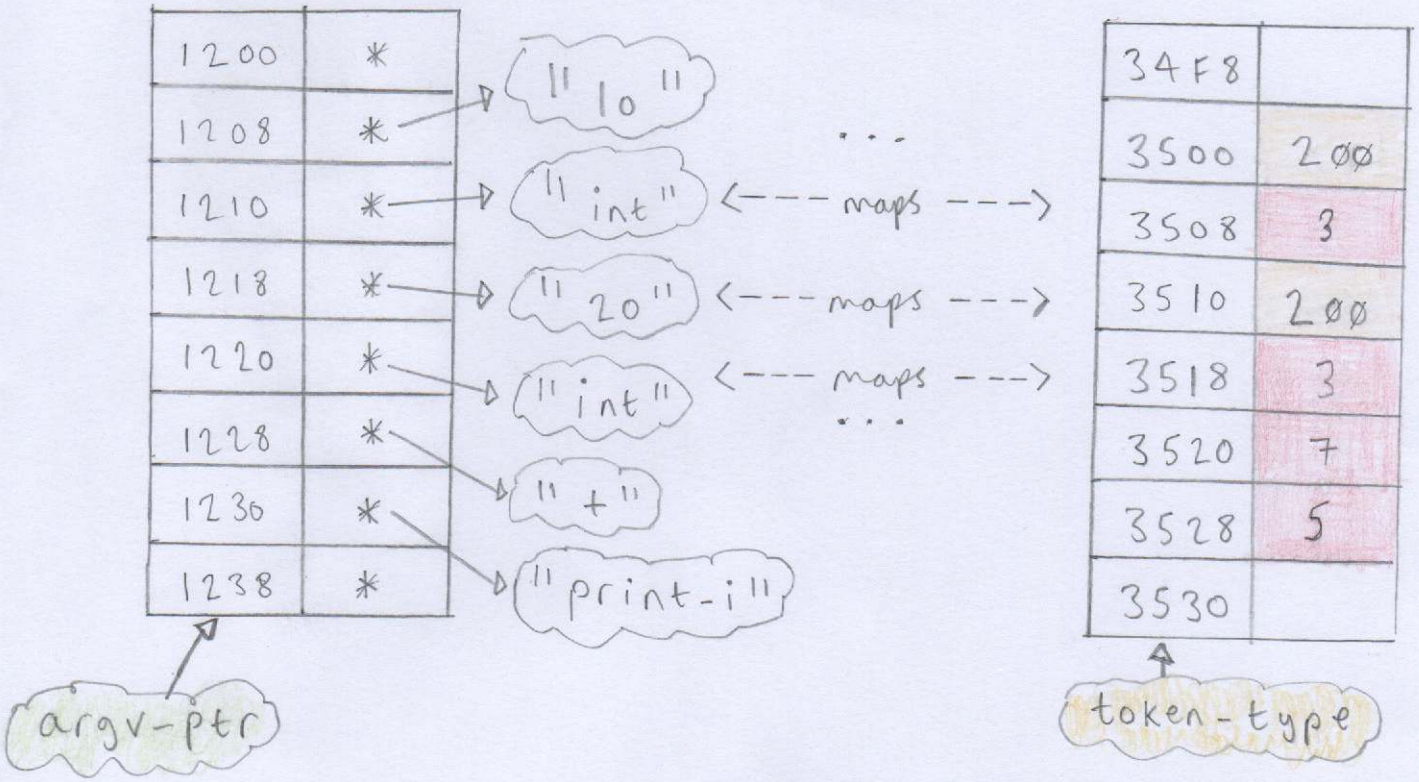
Before each token is processed, at the start of the program, we could scan over every single token and mark it with a byte. Any operator will have a number which the Hashset we spoke about before generates and any operand will be marked as 0x(8 (200):

operators : 0 → 125 and the number is the function location in [map-funcs + (n) * 0x08]

Operands : 200

For this code, we can see the matching token-type.

```
10 int 20 int + print-i
```



When processing the tokens, we can now just look at token-type and decide to push on to our rts if it's value == 200 (0x(8) or call the function if it's below <200 (0x(8). This number will be used to call [map-funcs + (n) * 0x08].

When Testing out my Program on Fibonacci, all permutations and all subset permutations, I noticed the times recorded were very slow. One of the reasons was due to creating and retrieving Variables.

Each time a Variable is created, the dict stack is scanned, this means many strcmp's, then a malloc to store the Variable and a push of the [Local-dict_tos] pointer.

Everytime we want to peek a variable, we scan all Variables and carry out strcmp's!

When we end a function, any Local Variables are removed using free library call and the [Local-dict-tos] is popped.

Compared to Languages such as C, Local Vars are pushed onto the stack and then retrieved by their offset n, [rbp -n*8], assuming each Var is placed +8 bytes from the previous one. Once the function ends, rbp and rsp are repositioned so the Local Variables are simply ignored!

Comparing both implementations, mine is too slow, but I found it difficult to find an alternative.

Then I thought about attempting to copy rsp and rbp in nasm. I could place values (for Variables) onto the stack, set a marker on the [rts-tos] and use offsetts to peek and poke these values directly! The disadvantage to this is that you need to keep track of the Variables and their offsetts!

Once you finished with the base-pointer, just pop it off. We can have many of these base-pointers set up by pushing them on and off.

This was the basis of my fast pointers implementation.

setting up the stack for the base pointer was easy:

declaring: base_ptr : resq 0xff
base-ptr-tos : resq 0x01

during init: mov qword [base-ptr-tos], base-ptr
sub qword [base-ptr-tos], 0x01 * EIGHT

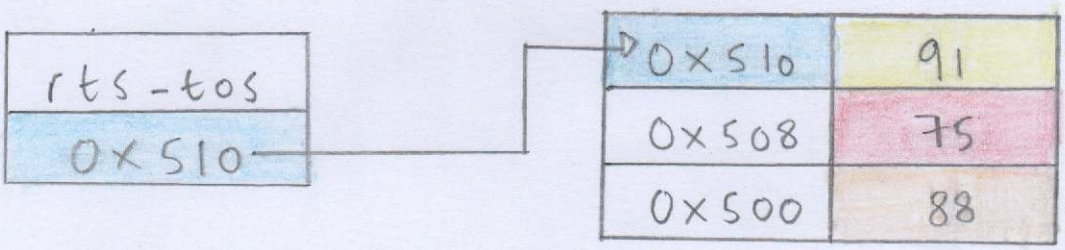
To place a base-ptr at the top of the rts : push-bp
To get the value from the base-ptr : get-bp
To remove the base-ptr : pop-bp

The code is simple to set up a base pointer:

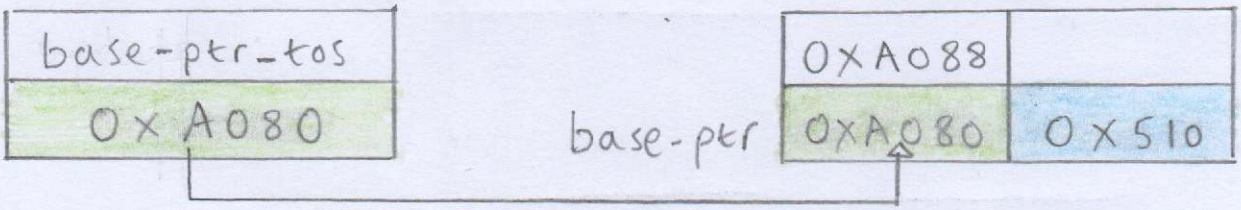
```
set-base-ptr:
        mov  rcx, base-ptr-tos
        add  qword [rcx], 0x01 * EIGHT
        mov  rcx, [rcx]
        mov  rdi, [rts_tos]
        mov  [rcx], rdi
```

Here's a simple example, we have 3 scores for maths 88, english 75 and science 91.

88 int 75 int 91 int push-bp



get-bp [peek] pushes the science value on top of the stack.
get-bp - 0x08 [peek] pushes English on top of the stack.
get-bp - 0x10 [peek] pushes Maths on top of the stack.



You can update the values by using [poke] too!